StoryMaker - Run wild with your ideas!

Author: Vikram Bhatt **Version:** 1.0.0

Description

The StoryMaker project is about using AI to build out short stories of different types ranging from fantasy to sci-fi to even more! I use OpenRouter to query the Arcee AI free model that is excellent at creative writing, story-telling, and much more! Additionally, the project is Object Oriented and uses classes to build out the story. It is a work in progress and there are many things I still want to add.

Getting Started

This project uses `uv` for dependency and virtual environment management. **Python 3.12 or higher is required.**

1. Install uv

macOS / Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Or via pip (any platform):

```
pip install uv
```

2. Create the virtual environment and install dependencies

```
uv sync
```

This reads `pyproject.toml`, creates a `.venv` folder, and installs all required packages automatically.

3. Set up your API key

Create a `.env` file in the project root and add your OpenRouter API key:

```
OPENROUTER_API=your_api_key_here
```

4. Run the project

Script (CLI):

```
uv run main.py --files <story_folder> <story_file> <history_folder> <history_file>
```

Streamlit app:

```
uv run streamlit run app.py
```

Assignment Points Covered

---

1. Classes

Location: `StoryMaker.py` lines 8–22 · `StoryHelper.py` lines 15–19

Two classes are defined: `StoryMaker` is the core AI generation class with a full class-level docstring listing its attributes. `StoryHelper` extends it and is defined in its own file.

```
8  class StoryMaker:
9      """
10     An AI-powered story generation class that interfaces with language models via OpenRouter.
11
12     Manages conversation history, model configuration, and HTTP connection lifecycle.
13     Supports both standard and streaming responses, and can be used as a context manager
14     to ensure the HTTP client is properly closed after use.
15
16     Attributes:
17         url (str): The OpenRouter API base URL.
18         main_model (str): The primary model used for generation.
19         fallback_models (list[str]): Fallback models if the primary is unavailable.
20         basic_prompt (str): Default user prompt when none is provided.
21         init_sys_prompt (str): Default system prompt applied when no custom prompt is given.
22     """
23
24     # Base URL
25     url = "https://openrouter.ai/api/v1"
26
27     # Models to use and fallbacks
28     main_model = "arcee-ai/trinity-large-preview:free"
29     __fallback_models = ["deepseek/deepseek-r1-0528:free", "meta-llama/llama-3.3-70b-instruct:free"]
30
```

```
15 class StoryHelper(StoryMaker):
16
17     def __init__(self):
18         self.__story_path = Path("story_inputs")
19         self.__image_path = self.__story_path / "posters"
20
```

---

2. Instances

Location: `main.py` lines 93–96 · `app.py` lines 18–21

`StoryMaker()` is instantiated inside `main()` based on user input — either with a custom system prompt or with the default. In `app.py`, `StoryHelper()` is instantiated inside a cached factory function.

```python
# ── StoryHelper Instance ──────────────────────────────────────────
# @st.cache_resource creates ONE shared instance for the entire session.
# The same StoryHelper (and its underlying HTTP client) is reused on every
# Streamlit rerun rather than creating a new object each time.
@st.cache_resource
def get_story_helper():
    return StoryHelper()

helper = get_story_helper()
```

```python
while condition_num != 1:
    # Initialize system prompt and make sure it follows order.
    sys_prompt = ask(
        qn="Do you have your own system prompt or do you want to use the default (d for default else type your answer).",
        err="Please enter 'd' or a system prompt that is descriptive!",
        err_check_type="System Prompt"
    )
    print("Perfect! You have chosen the system prompt:")
    if sys_prompt == "d":
        print(StoryMaker.get_default_system_prompt())
        story_maker = StoryMaker()
    else:
        print(sys_prompt)
        story_maker = StoryMaker(sys_prompt)
    print("--------------------")
```

---

3. Class Attributes

Location: `StoryMaker.py` lines 24–35

Five class-level attributes are defined directly on the `StoryMaker` class body (not inside `__init__`): `url`, `main_model`, `__fallback_models`, `basic_prompt`, and `init_sys_prompt`. These are shared across all instances.

```python
    # Base URL
    url = "https://openrouter.ai/api/v1"

    # Models to use and fallbacks
    main_model = "arcee-ai/trinity-large-preview:free"
    __fallback_models = ["deepseek/deepseek-r1-0528:free", "meta-llama/llama-3.3-70b-instruct:free"]

    # system and prompts
    basic_prompt = "Write a one paragraph story about a hero who goes on a journey, meets alies, gets stronger, finds weapons and t
    init_sys_prompt = "You are a master storyteller known for bestselling and critically acclaimed novels. Your writing is cinemati
                       Show, don't tell. Use sensory detail, internal monologue, and layered description. Build tension naturally.
                        Avoid generic phrasing, shallow description, and mechanical structure."
```

---

4. Instance Attributes

Location: `StoryMaker.py` lines 49–53 · `StoryHelper.py` lines 18–19

`StoryMaker.__init__` sets four instance attributes: `self.temp`, `self.max_tokens`, `self.stream_result`, and `self.__preserve_convo`. `StoryHelper.__init__` sets `self.__story_path` and `self.__image_path` as `Path` objects unique to each instance.

```
49          # make sure some values are set.
50          self.temp = 1
51          self.max_tokens = 5000
52          self.stream_result = False
53          self.__preserve_convo = []
54
```

```
15  class StoryHelper(StoryMaker):
16
17      def __init__(self):
18          self.__story_path = Path("story_inputs")
19          self.__image_path = self.__story_path / "posters"
20
```

---

5. Mutable / Immutable Attributes

Location: `StoryMaker.py` lines 25–35 (class) · lines 49–53 (instance)

Class attributes `url`, `main_model`, and `basic_prompt` are strings — **immutable** in Python. Instance attribute `self.__preserve_convo` is a list — **mutable**, and is explicitly modified throughout the class via `.append()` and `.clear()`.

```python
    # Base URL
    url = "https://openrouter.ai/api/v1"

    # Models to use and fallbacks
    main_model = "arcee-ai/trinity-large-preview:free"
    __fallback_models = ["deepseek/deepseek-r1-0528:free", "meta-llama/llama-3.3-70b-instruct:free"]

    # system and prompts
    basic_prompt = "Write a one paragraph story about a hero who goes on a journey, meets alies, gets stronger, finds weapons and t
    init_sys_prompt = "You are a master storyteller known for bestselling and critically acclaimed novels. Your writing is cinemati
                       Show, don't tell. Use sensory detail, internal monologue, and layered description. Build tension naturally.
                        Avoid generic phrasing, shallow description, and mechanical structure."

    def __init__(self, system_prompt:str=""):
        """
        Initializes the StoryMaker with default settings.

        The HTTP client is not created here. It is created lazily the first time
        generate() is called.

        Args:
            system_prompt (str): A custom system prompt to guide the model's behavior.
                If left empty, the default storytelling prompt is used.
        """

        # make sure some values are set.
        self.temp = 1
        self.max_tokens = 5000
        self.stream_result = False
        self.__preserve_convo = []
```

---

6. Hashable Objects

Location: `StoryHelper.py` lines 7–13 · `app.py` lines 62–67 · `app.py` line 205

`StoryRecord` is a `namedtuple` — immutable and therefore hashable. This allows it to be used directly as a **dictionary key** in `st.session_state.generated_stories`, which maps each story archetype to its generated text. Plain dicts or lists cannot be used as dict keys.

```python
7   # A lightweight, immutable record for a single story archetype.
8   # Being a namedtuple makes StoryRecord hashable, so instances can be used
9   # as dictionary keys or stored in sets — see app.py's generated_stories dict.
10  StoryRecord = namedtuple(
11      'StoryRecord',
12      ['id', 'protagonist', 'description', 'setting', 'plot', 'conflict', 'theme', 'point_of_view']
13  )
14
```

```python
62  # ── Session State ──────────────────────────────────────────────
63  # generated_stories maps StoryRecord → generated text.
64  # StoryRecord is a namedtuple (immutable + hashable), so it can be used as a
65  # dictionary key — this is the hashability use case.
66  if "generated_stories" not in st.session_state:
67      st.session_state.generated_stories = {}
```

```python
199                     # Persist the result so it survives the next rerun, then
200                     # rerun to replace the live stream with a stable text area.
201                     st.session_state[key_result] = result
202                     st.session_state[key_show]   = True
203                     # Store in generated_stories using StoryRecord as the key.
204                     # This works because StoryRecord is hashable (namedtuple).
205                     st.session_state.generated_stories[story] = result
206                     st.rerun()
```

---

7. Magic Methods

Location: `StoryMaker.py` lines 37–66 · lines 281–302

Four dunder methods are implemented in `StoryMaker`:

- `__init__` (lines 37–66): sets up instance state and conversation history
- `__del__` (lines 281–286): safety-net cleanup when object is garbage collected
- `__enter__` (lines 289–291): enables use as a `with` statement, returns `self`
- `__exit__` (lines 294–302): guaranteed cleanup when the `with` block exits

```
281     def __del__(self):
282         """Safety net to close the HTTP client when the object is garbage collected.
283         Clears preserved conversation with model to clear unused space."""
284         if hasattr(self, 'client'):
285             self.client.close()
286         self.__preserve_convo.clear()
287
288
289     def __enter__(self):
290         """Enables use as a context manager. Returns the StoryMaker instance."""
291         return self
292
293
294     def __exit__(self, _exc_type, _exc_val, _exc_tb):
295         """Closes the HTTP client when exiting a context manager block.
296         Clears preserved conversation with model to clear unused space.
297
298         Does nothing if the client was never created.
299         """
300         if hasattr(self, 'client'):
301             self.client.close()
302         self.__preserve_convo.clear()
```

---

8. Docstrings

Location: `StoryMaker.py` lines 9–22 · lines 38–47 · `StoryHelper.py` lines 65–76

Google-style docstrings are used throughout. The class has an `Attributes` section. Methods have `Args`, `Returns`, and `Yields` sections where applicable. One-liners are used for simple methods (e.g., `close()`).

```python
    def get_helper_story(self, story_id: int) -> dict:
        """
        Return a single story type dict by its 1-based ID.

        Args:
            story_id (int): The 1-based ID of the story (matches the "id" field
                in story_types.json).

        Returns:
            dict: The story type entry for the given ID.
        """
        self.__check_attr("helpers")
        return self.helpers[story_id - 1]
```

---

9. Inheritance

Location: `StoryHelper.py` line 15 · lines 171–174

`StoryHelper` inherits from `StoryMaker` using `class StoryHelper(StoryMaker)`. `super().__init__(system_prompt)` is called inside `generate_story()` rather than in `__init__` — this intentionally defers parent initialisation until a system prompt is available at generation time.

`class StoryHelper(StoryMaker):`

`super().__init__(system_prompt)`

---

10. Composition

Location: `StoryMaker.py` lines 69–71 · lines 154–155

`StoryMaker` uses **composition** by holding an `OpenAI` client object (`self.client`) as an instance attribute. The client is created lazily via `__create_client()` and is only instantiated when `generate()` or `stream_generate()` is first called.

```python
    def __create_client(self):
        """Creates and assigns the OpenAI HTTP client. Called lazily by
generate()."""
        self.client = OpenAI(base_url=self.url, api_key=open_ai_key)
```

---

11. *args

Location: `StoryHelper.py` line 150 · lines 178–185

`generate_story()` accepts `*args` to receive a variable number of story detail strings (protagonist, description, setting, etc.) from the Streamlit app. The args are then iterated with `zip(labels, args)` to assemble a structured prompt.

```python
def generate_story(self, system_prompt: str, *args):
    """
    Initialize StoryMaker with a system prompt and generate a story.

    Calling super().__init__() here (rather than in StoryHelper.__init__)
    lets us inject the user-selected system prompt at generation time.
    Each call starts a fresh StoryMaker conversation, so successive calls
    are independent of one another.

    Args:
        system_prompt (str): The full system prompt text to pass to
            StoryMaker (e.g. the "system_prompt" field from a prompt dict).
        *args: Story detail strings passed in from the Streamlit app,
            expected in this order:
                protagonist, description, setting, plot,
                conflict, theme, point_of_view
            These are assembled into a structured prompt for the model.

    Returns:
        str: The generated story text from StoryMaker.
    """
    # Re-initialize StoryMaker fresh with the chosen system prompt.
    # stream_generate() handles streaming internally, so turn_on_streaming()
    # is not needed here.
    super().__init__(system_prompt)

    # Assemble the story fields into a structured prompt for the model.
    # The labels match the order the Streamlit app passes the args.
    labels = [
        "Protagonist", "Description", "Setting",
        "Plot", "Conflict", "Theme", "Point of View"
    ]
    prompt_lines = ["Write a story with the following details:"]
    for label, value in zip(labels, args):
        prompt_lines.append(f"- {label}: {value}")
    prompt = "\n".join(prompt_lines)

    # yield from turns generate_story() into a generator, so the caller
    # (e.g. st.write_stream) receives chunks as they arrive from the model.
    yield from self.stream_generate(prompt)
```

---

12. **kwargs

Location: `StoryMaker.py` lines 191–219 · `main.py` lines 39–40

`StoryMaker.update()` accepts `**kwargs` and iterates over them to build an update instruction for the model. `update_storyMaker()` in `main.py` passes `**updates` through to `story.update()`.

---

13. Default Arguments

Location: `StoryMaker.py` lines 37, 164, 223

Three methods demonstrate default argument values:

- `__init__(self, system_prompt:str="")` — defaults to the built-in system prompt
- `generate(self, prompt:str="")` — defaults to `basic_prompt`
- `get_convo_history(self, pretty:bool=False)` — defaults to raw list output

---

14. Positional Arguments

Location: `main.py` lines 36–37 · lines 131–132

`generate_storyMaker(story, prompt)` defines and calls arguments positionally. `story_maker` and `prompt` are passed in order without keyword labels at the call site on line 132.

```
def generate_storyMaker(story:StoryMaker, prompt:str=""):
    return story.generate(prompt)
```

---

15. Keyword Arguments

Location: `main.py` lines 85–90 · line 137

`ask()` is called with all three parameters passed by keyword name (`qn=`, `err=`, `err_check_type=`), making the call site self-documenting. On line 137, `close_storyMaker` is called with `pretty=True` as a keyword argument.

```
def ask(qn:str, err:str, err_check_type:str):
    ans = input(qn + "\n Answer:")
    while not check_correct(err_check_type, ans):
        print(err)
        ans = input(qn + "\n Answer:")
    return ans
```

---

16. Decorators

Location: `main.py` lines 28–44 · `StoryMaker.py` lines 305–308 · `app.py` lines 17–19, 28–31

Four decorator types are demonstrated:

- **Custom decorator** with `@functools.wraps` (`main.py` lines 28–34) — captures conversation history before closing
- `@get_conversation_storyMaker` applied to `close_storyMaker` (`main.py` lines 42–44)
- `@classmethod` on multiple utility methods (`StoryMaker.py` lines 305–308)
- `@st.cache_resource` and `@st.cache_data` in `app.py`

```python
def get_conversation_storyMaker(func):
    @functools.wraps(func)
    def wrapper(story:StoryMaker, pretty:bool=True):
        history = story.get_convo_history(pretty)
        func(story)
        return history
    return wrapper

@get_conversation_storyMaker
def close_storyMaker(story:StoryMaker):
    story.close()
```

---

17. Generators

Location: `StoryMaker.py` lines 102–136 · lines 139–161 · `StoryHelper.py` line 189

`__stream_chat()` uses `yield` to send each text chunk from the API to the caller as it arrives, then appends the complete response to history after the loop. `stream_generate()` uses `yield from` to delegate to `__stream_chat()`. `generate_story()` in `StoryHelper` also uses `yield from` to chain the whole pipeline, enabling `st.write_stream()` to consume it directly.

```python
def __stream_chat(self):
        """
        Generator version of __chat() for streaming output.

        Makes the same API call as __chat() but always streams, yielding each
        text chunk as it arrives instead of printing it. After all chunks have
        been yielded, the complete response is appended to conversation history
        exactly like __chat() does, keeping the history consistent.

        Yields:
            str: Individual text chunks from the model as they arrive.
        """
        response = self.client.chat.completions.create(
            model=self.main_model,
            messages=self.__preserve_convo,
            extra_body={
                "models": self.__fallback_models
            },
            max_tokens=self.max_tokens,
            stream=True,                # always stream in this path
            temperature=self.temp
        )

        complete_response = ""
        for chunk in response:
            if chunk.choices[0].delta.content is not None:
                content = chunk.choices[0].delta.content
                complete_response += content
```

```
                yield content            # send chunk to the caller

        # Append the full assembled response to history once streaming is done
        self.__preserve_convo.append({
            "role": "assistant",
            "content": complete_response,
        })


    def stream_generate(self, prompt: str = ""):
        """
        Generator version of generate() that yields text chunks for streaming.

        Intended for use with Streamlit's st.write_stream() or any other caller
        that consumes a generator. Creates the HTTP client on first call if it
        does not already exist.

        Args:
            prompt (str): The story prompt to send to the model. If empty, the
                default basic_prompt is used.

        Yields:
            str: Individual text chunks from the model as they arrive.
        """
        if not hasattr(self, 'client'):
            self.__create_client()

        message = {"role": "user", "content": prompt if prompt else self.basic_prompt}
        self.__preserve_convo.append(message)

        # Delegate to __stream_chat() which handles the streaming loop
        yield from self.__stream_chat()
```

---

18. Scripts

Location: `main.py` lines 7–8, 22–26, 164–165

`main.py` is structured as a proper Python script: `argparse` is configured at module level, the entry point is guarded with `if __name__ == "__main__":`, and `main()` contains all runtime logic.

---

19. Paths

Location: `main.py` lines 5, 11–12, 14–18, 140–147 · `StoryHelper.py` lines 5, 18–19, 24, 43, 115

`pathlib.Path` is used in both scripts. In `main.py`, paths are built with the `/` operator, checked with `.exists()`, and files are opened with `.open("a")`. In `StoryHelper.py`, all JSON and image paths are composed the same way, replacing all f-string path concatenation.

```
from pathlib import Path
```

```
# story file
with story_path.open("a") as f:
    f.write(initial_story)
    f.write("")

# history file
with history_path.open("a") as f:
    f.write(conversation)
    f.write("")
```

```
def ensure_files(files):
    story_path   = Path(files[0]) / files[1]
    history_path = Path(files[2]) / files[3]

    if not story_path.exists():
        raise FileNotFoundError("Story file not found. Please check the path or
create the story.")

    if not history_path.exists():
        raise FileNotFoundError("History file not found. Please check the path or
create the story.")

    return story_path, history_path
```

---

20. Collections

Location: `StoryHelper.py` lines 2, 7–13 · lines 26–38

`namedtuple` from the `collections` module is used to define `StoryRecord` — a lightweight, immutable record for a story archetype. `__load_helpers()` converts each raw JSON dict into a `StoryRecord` instance using a list comprehension, flattening the nested `characters` sub-dict.

```python
from collections import namedtuple
from PIL import Image
import json
from pathlib import Path

# A lightweight, immutable record for a single story archetype.
# Being a namedtuple makes StoryRecord hashable, so instances can be used
# as dictionary keys or stored in sets — see app.py's generated_stories dict.
StoryRecord = namedtuple(
    'StoryRecord',
    ['id', 'protagonist', 'description', 'setting', 'plot', 'conflict', 'theme', 'point_of_view']
)

class StoryHelper(StoryMaker):

    def __init__(self):
        self.__story_path = Path("story_inputs")
        self.__image_path = self.__story_path / "posters"


    def __load_helpers(self):
        """Load story types from story_types.json into self.helpers as StoryRecord instances."""
        with open(self.__story_path / "story_types.json") as file:
            raw = json.load(file)
        self.helpers = [
            StoryRecord(
                id=story['id'],
                protagonist=story['characters']['protagonist'],
                description=story['characters']['description'],
                setting=story['setting'],
                plot=story['plot'],
                conflict=story['conflict'],
                theme=story['theme'],
                point_of_view=story['point_of_view'],
            )
            for story in raw
        ]
```

---

21. Iterables

Location: `StoryHelper.py` lines 183–184 · `StoryMaker.py` lines 241–244 · `app.py` lines 50, 245–255

Iterables are used throughout: `zip(labels, args)` pairs field labels with values in `generate_story()`; a `for` loop iterates over `__preserve_convo` in `get_convo_history()`; a dict comprehension builds `story_lookup` in `app.py`; and `dict.items()` is iterated in the Created Stories tab.

```python
        prompt_lines = ["Write a story with the following details:"]
        for label, value in zip(labels, args):
            prompt_lines.append(f"- {label}: {value}")
        prompt = "\n".join(prompt_lines)
```

```python
    # ── Created Stories ──────────────────────────────────────────────
    # Iterates over generated_stories, which is keyed by StoryRecord instances.
    # Because StoryRecord is a namedtuple it is hashable and can serve as a
    # dict key — each entry maps a story archetype to its generated text.
    st.markdown("### Stories generated this session")

    if not st.session_state.generated_stories:
        st.info("No stories generated yet. Head to **Browse** and hit ✍️ **Generate Story**!")
    else:
        for record, text in st.session_state.generated_stories.items():
            with st.expander(f"**{record.protagonist}** — {record.setting}", expanded=False):
                st.markdown(f"*{record.description}*")
                st.markdown("---")
                st.text_area(
                    label="story_output",
                    value=text,
                    height=400,
                    disabled=True,
                    label_visibility="collapsed"
                )

```

---

22. Pass by Reference vs Copy

Location: `StoryMaker.py` lines 236–237

`get_convo_history(pretty=False)` returns `list(self.__preserve_convo)` — a **shallow copy** — rather than the list itself. Returning the list directly would pass a reference to the internal state, allowing the caller to corrupt conversation history with appends or removals.

```python
def get_convo_history(self, pretty:bool=False):
    """
    Returns the current conversation history.

    Args:
        pretty (bool): If False, returns a copy of the raw list of message
            dictionaries. If True, returns a human-readable formatted string
            with each message separated by a divider line.

    Returns:
        list[dict] | str: The conversation history as a list of dicts when
            pretty=False, or a formatted string when pretty=True.
    """
    if not pretty:
        return list(self.__preserve_convo)

    divider = "----------------------------------------"
    result = ""
    for message in self.__preserve_convo:
        result += f"{divider}\n"
        result += f"role: {message['role']}\n"
        result += f"content: {message['content']}\n"
    result += divider
    return result
```

---

23. Deep / Shallow Copy

Location: `StoryMaker.py` lines 236–237

`list(self.__preserve_convo)` creates a **shallow copy** — a new list object, but the dict elements inside still point to the same objects as the original. This is sufficient here since callers receiving the history are not expected to mutate the message dicts. A `copy.deepcopy()` would fully isolate the nested dicts but is not needed for this use case.

```python
def get_convo_history(self, pretty:bool=False):
    """
    Returns the current conversation history.

    Args:
        pretty (bool): If False, returns a copy of the raw list of message
            dictionaries. If True, returns a human-readable formatted string
            with each message separated by a divider line.

    Returns:
        list[dict] | str: The conversation history as a list of dicts when
            pretty=False, or a formatted string when pretty=True.
    """
    if not pretty:
        return list(self.__preserve_convo)

    divider = "----------------------------------------"
    result = ""
    for message in self.__preserve_convo:
        result += f"{divider}\n"
        result += f"role: {message['role']}\n"
        result += f"content: {message['content']}\n"
    result += divider
    return result
```

---

24. Menus

Location: `main.py` lines 46–67 · lines 83–105

`check_correct(question, answer)` validates input differently depending on the question type. `ask(qn, err, err_check_type)` wraps `input()` in a `while` loop that re-prompts until a valid answer is given. The `main()` function presents two numbered menus using these helpers.

```python
while condition_num != 1:
    # Initialize system prompt and make sure it follows order.
    sys_prompt = ask(
        qn="Do you have your own system prompt or do you want to use the default (d for default else type your answer).",
        err="Please enter 'd' or a system prompt that is descriptive!",
        err_check_type="System Prompt"
    )
    print("Perfect! You have chosen the system prompt:")
    if sys_prompt == "d":
        print(StoryMaker.get_default_system_prompt())
        story_maker = StoryMaker()
    else:
        print(sys_prompt)
        story_maker = StoryMaker(sys_prompt)
    print("--------------------")

    # Build out generator prompt
    print("Now here is a menu, please select from the options!")
    ask_to_do = ask(
        qn="1. Generate new prompt.\n2. Close.",
        err="Please only input either a 1, or 2. Thank you!",
        err_check_type="Question"
    )

    if ask_to_do == "2":
        condition_num = 1
        print("Thank you! Enjoy your day!")
        print("--------------------")

    elif ask_to_do == "1":
        print("You have selected to generate a new prompt!")
        print("--------------------")
```

---

25. argparse

Location: `main.py` lines 3, 7–8, 22–26

`argparse.ArgumentParser` is used to accept `--files` from the command line. The argument takes exactly 4 values (`nargs=4`) — story folder, story filename, history folder, history filename. `parse_args()` is called at module level so arguments are available throughout the script.

argparse1

---

26. APIs

Location: `StoryMaker.py` lines 1–6, 69–71, 83–99, 114–136

The OpenRouter API is called via the OpenAI-compatible SDK. The client is initialised lazily with a base URL and API key. Both `__chat()` (non-streaming) and `__stream_chat()` (streaming) send the full conversation history to the model and handle the response, including fallback models via `extra_body`.

```python
from openai import OpenAI
```

```python
    def __create_client(self):
        """Creates and assigns the OpenAI HTTP client. Called lazily by generate()."""
        self.client = OpenAI(base_url=self.url, api_key=open_ai_key)


    def __chat(self):
        """
        Sends the current conversation to the model and appends the response to history.

        Handles the non-streaming mode.

        Returns:
            str: The complete response text from the model.
        """
        response = self.client.chat.completions.create(
            model=self.main_model,
            messages=self.__preserve_convo,
            extra_body={
                "models": self.__fallback_models
            },
            max_tokens=self.max_tokens,
            stream=self.stream_result,
            temperature=self.temp
        )

        self.__preserve_convo.append({
                "role": "assistant",
                "content": response.choices[0].message.content,
            }
        )
        return response.choices[0].message.content
```

---

27. .env

Location: `StoryMaker.py` lines 1, 5–6

`python-dotenv`'s `dotenv_values(".env")` loads the `.env` file at module level and extracts the `OPENROUTER_API` key. This keeps the API key out of source code and out of version control.

```python
5   config = dotenv_values(".env")
6   open_ai_key = config["OPENROUTER_API"]
```

---

28. README.md

Location: `README.md` lines 1–144

The README covers: project title and description, a Features section with three screenshots, full Getting Started instructions for macOS/Linux/Windows using `uv`, file structure with annotations, a dependencies table, and a Next Steps checklist.

```md
# StoryMaker - Run wild with your ideas!

**Author:** Vikram Bhatt
**Version:** 1.0.0

---

## Description

The StoryMaker project is about using AI to build out short stories of
more! I use OpenRouter to query the Arcee AI free model that is excell
Additionally, the project is Object Oriented and uses classes to build
things I still want to add.
```

---

29. pyproject.toml

Location: `pyproject.toml` lines 1–17

All project metadata and dependencies are declared in `pyproject.toml` using the `uv` standard. Every package that is directly imported (`openai`, `python-dotenv`, `streamlit`, `pillow`, `pandas`) is listed as a direct dependency with a minimum version constraint.

> VIKRAM - NEED TO ADD IMAGE. Python file pyproject.toml lines 1-17

```toml
[project]
name = "StoryMaker"
version = "0.1.0"
description = "StoryMaker: Run wild with you ideas! The StoryMaker project is about using AI to build out short stories of differen
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "openai>=2.21.0",
    "pandas>=2.2.0,<3.14.3",
    "pillow>=10.0.0",
    "python-dotenv>=1.2.1",
    "streamlit>=1.30",
]
```

---

30. Object Scope

Location: `StoryMaker.py` lines 24–35 (class scope) · lines 49–53 (instance scope) · `StoryHelper.py` lines 7–13 (module scope)

Three scopes are demonstrated: `StoryRecord` is defined at **module scope** in `StoryHelper.py`; `url`, `main_model`, etc. are **class-scope** attributes shared by all instances; `self.temp`, `self.__preserve_convo`, etc. are **instance-scope** attributes unique per object. Private name mangling (`__preserve_convo`, `__story_path`) is used to restrict access.

```python
# A lightweight, immutable record for a single story archetype.
# Being a namedtuple makes StoryRecord hashable, so instances can be used
# as dictionary keys or stored in sets — see app.py's generated_stories dict.
StoryRecord = namedtuple(
    'StoryRecord',
    ['id', 'protagonist', 'description', 'setting', 'plot', 'conflict', 'theme', 'point_of_view']
)
```

```python
    # Base URL
    url = "https://openrouter.ai/api/v1"

    # Models to use and fallbacks
    main_model = "arcee-ai/trinity-large-preview:free"
    __fallback_models = ["deepseek/deepseek-r1-0528:free", "meta-llama/llama-3.3-70b-instruct:free"]

    # system and prompts
    basic_prompt = "Write a one paragraph story about a hero who goes on a journey, meets alies, gets stronger, finds weapons and t
    init_sys_prompt = "You are a master storyteller known for bestselling and critically acclaimed novels. Your writing is cinemati
                       Show, don't tell. Use sensory detail, internal monologue, and layered description. Build tension naturally.
                        Avoid generic phrasing, shallow description, and mechanical structure."

    def __init__(self, system_prompt:str=""):
        """
        Initializes the StoryMaker with default settings.

        The HTTP client is not created here. It is created lazily the first time
        generate() is called.

        Args:
            system_prompt (str): A custom system prompt to guide the model's behavior.
                If left empty, the default storytelling prompt is used.
        """

        # make sure some values are set.
        self.temp = 1
        self.max_tokens = 5000
        self.stream_result = False
        self.__preserve_convo = []
```

---

31. For Loops

Location: `StoryMaker.py` lines 213–214, 241–244 · `StoryHelper.py` lines 183–184 · `app.py` lines 163–164, 245–255

For loops are used throughout: iterating over `kwargs.items()` to build update messages, iterating over `__preserve_convo` to format history, `zip(labels, args)` to assemble prompts, and iterating over `generated_stories.items()` to render the Created Stories tab.

```python
        # Make the message
        message = {"role":"user", "content": "Update the story you have created with the following parameters:\n"}
        for key, value in kwargs.items():
            message["content"] += f"{key}: {value}\n"
        self.__preserve_convo.append(message)
```

```python
        labels = [
            "Protagonist", "Description", "Setting",
            "Plot", "Conflict", "Theme", "Point of View"
        ]
        prompt_lines = ["Write a story with the following details:"]
        for label, value in zip(labels, args):
            prompt_lines.append(f"- {label}: {value}")
        prompt = "\n".join(prompt_lines)
```

---

32. While Loops

Location: `main.py` lines 64–67 · lines 83–161

Two while loops are present: `ask()` uses `while not check_correct(...)` to re-prompt the user until their input is valid. `main()` uses `while condition_num != 1` as the main application loop — set `condition_num = 1` to exit, otherwise the loop continues for another story generation cycle.

```
62 def ask(qn:str, err:str, err_check_type:str):
63     ans = input(qn + "\n Answer:")
64     while not check_correct(err_check_type, ans):
65         print(err)
66         ans = input(qn + "\n Answer:")
67     return ans
```

```
while condition_num != 1:
    # Initialize system prompt and make sure it follows order.
    sys_prompt = ask(
        qn="Do you have your own system prompt or do you want to use the default (d for default else type your answer).",
        err="Please enter 'd' or a system prompt that is descriptive!",
        err_check_type="System Prompt"
    )
    print("Perfect! You have chosen the system prompt:")
    if sys_prompt == "d":
        print(StoryMaker.get_default_system_prompt())
        story_maker = StoryMaker()
    else:
        print(sys_prompt)
        story_maker = StoryMaker(sys_prompt)
    print("--------------------")

    # Build out generator prompt
    print("Now here is a menu, please select from the options!")
    ask_to_do = ask(
        qn="1. Generate new prompt.\n2. Close.",
        err="Please only input either a 1, or 2. Thank you!",
        err_check_type="Question"
    )

    if ask_to_do == "2":
        condition_num = 1
        print("Thank you! Enjoy your day!")
        print("--------------------")

    elif ask_to_do == "1":
        print("You have selected to generate a new prompt!")
        print("--------------------")
```